Get data from the internet

June 16, 2026 5:00 PM

https://developer.android.com/codelabs/basic-android-kotlin-compose-getting-data-internet#0

Get data from the internet

About this codelab

Last updated May 10, 2024

Written by Google Developers Training team

1. Before you begin

Most Android apps in the market connect to the internet to perform network operations, such as retrieving emails, messages, or other information from a backend server. Gmail, YouTube, and Google Photos are example apps that connect to the internet to display the user data.

In this codelab, you will use open source and community-driven libraries to build a data layer and get data from a backend server. This greatly simplifies fetching the data and also helps the app follow Android best practices, such as perform operations on a background thread. You will also display an error message if the internet is slow or unavailable, which will keep the user informed about any network connectivity issues.

Prerequisites

- Basic knowledge of how to create Composable functions.
- Basic knowledge of how to use Android architecture components `ViewModel`.
- Basic knowledge of how to use coroutines for long-running tasks.
- Basic knowledge of how to add dependencies in `build.gradle.kts`.

What you'll learn

- ✓ What a REST web service is.
- ✓ How to use the Retrofit library to connect to a REST web service on the internet and get a response.
- ✓ How to use the Serialization (kotlinx.serialization) library to parse the JSON response into a data object.

What you'll do

- Modify a starter app to make a web service API request and handle the response.
- Implement a data layer for your app using the Retrofit library.
- Parse the JSON response from the web service into your app's list of data objects with the *kotlinx.serialization*

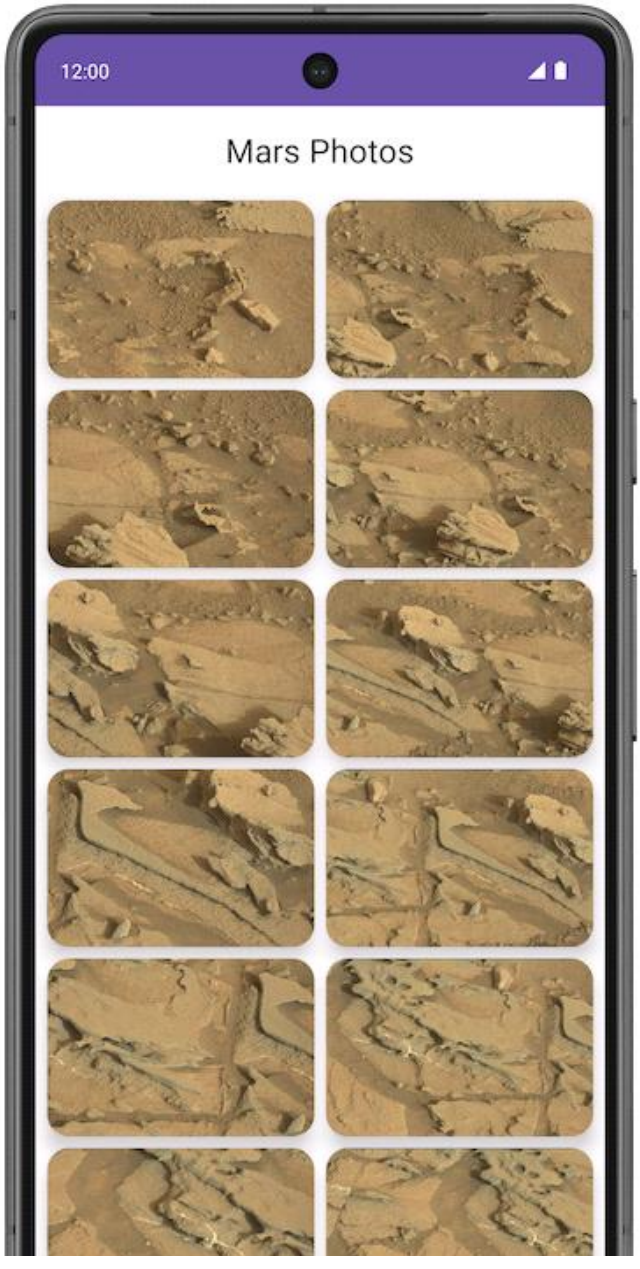
- Implement a data layer for your app using the Retrofit library.
- Parse the JSON response from the web service into your app's list of data objects with the *kotlinx.serialization* library, and attach it to the UI state.
- Use Retrofit's support for coroutines to simplify the code.

What you need

- A computer with Android Studio
- Starter code for the Mars Photos app

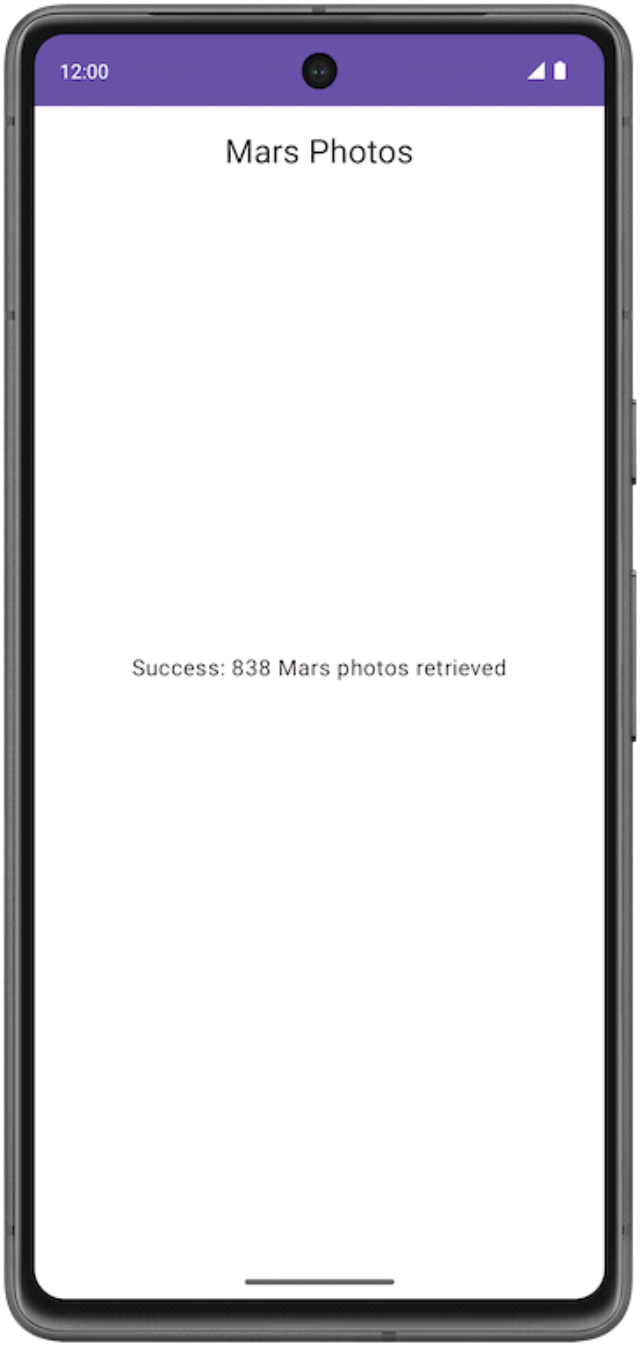
2. App overview

You work with the app named **Mars Photos**, which shows images of the Mars surface. This app connects to a web service to retrieve and display Mars photos. The images are real-life photos from Mars, captured from NASA's Mars rovers. The following image is a screenshot of the final app, which contains a grid of images.



Note: The preceding image is a screenshot of the final app that you will build at the end of this unit, after additional updates in later codelabs. The screenshot is shown in this codelab to give you a better idea of the overall app functionality.

The version of the app you build in this codelab won't have a lot of visual flash. This codelab focuses on the data layer part of the app to connect to the internet and download the raw property data using a web service. To ensure that the app correctly retrieves and parses this data, you can print the number of photos received from the backend server in a `Text` composable.



3. Explore the Mars Photos starter app

Download starter code

To get started, download the starter code:

Download zip

Alternatively, you can clone the GitHub repository for the code:

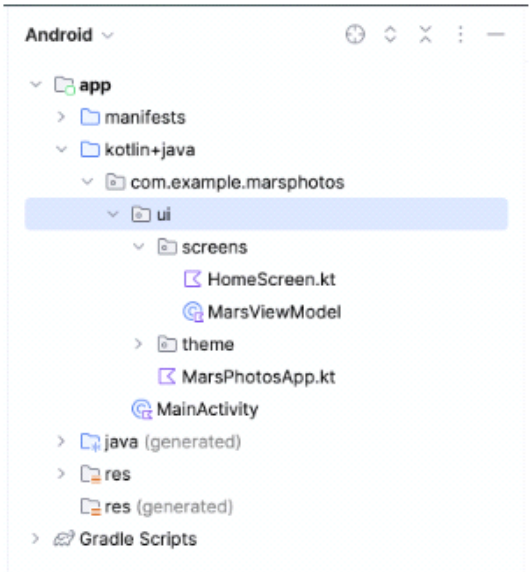
```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-tra
$ cd basic-android-kotlin-compose-training-mars-photos
$ git checkout starter
```

Note: The starter code is in the `starter` branch of the downloaded repository.

You can browse the code in the `Mars Photos` GitHub repository.

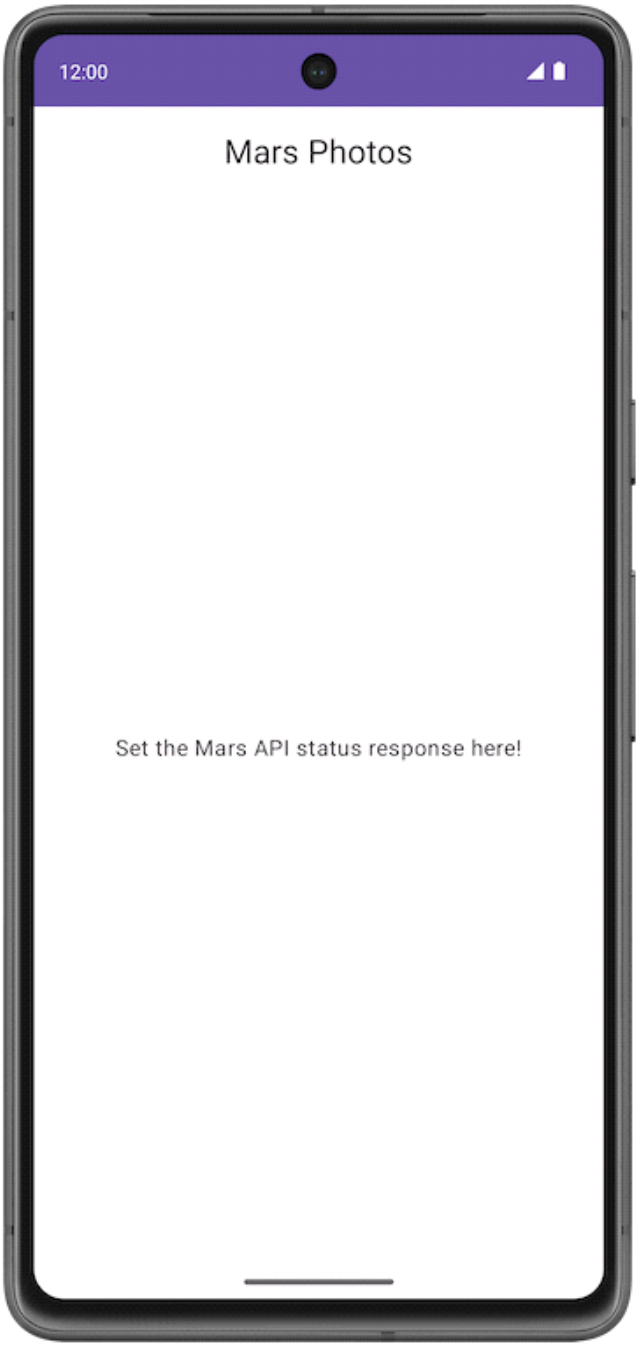
Run starter code

1. Open the downloaded project in Android Studio. The folder name of the project is `basic-android-kotlin-compose-training-mars-photos`.
2. In the **Android** pane, expand **app** > **kotlin + java**. Notice that the app has a package folder called `ui`. This is the UI layer of the app.



3. Run the app. When you compile and run the app, you see the following screen with placeholder text in the center. At the end of this codelab, you update this placeholder text with the number of photos retrieved.

3. Run the app. When you compile and run the app, you see the following screen with placeholder text in the center. At the end of this codelab, you update this placeholder text with the number of photos retrieved.



Starter code walkthrough

In this task, you familiarize yourself with the structure of the project. The following lists provide a walkthrough of the important files and folders in the project.

`ui\MarsPhotosApp.kt`:

- This file contains the composable, `MarsPhotosApp`, which displays the contents on the screen, such as the top app bar and the `HomeScreen` composable. The placeholder text in the previous step is displayed in this composable.
- In the next codelab, this composable displays the data received from the Mars photos backend server.

`screens\MarsViewModel.kt`:

- This file is the corresponding view model for the `MarsPhotosApp`.

`screens\MarsViewModel.kt`:

- This file is the corresponding view model for the `MarsPhotosApp`.
- This class contains a `MutableState` property named `marsUiState`. Updating the value of this property updates the placeholder text displayed on the screen.
- The `getMarsPhotos()` method updates the placeholder response. Later in the codelab, you use this method to display the data fetched from the server. The goal for this codelab is to update the `MutableState` within the `ViewModel` using data you get from the internet.

`screens\HomeScreen.kt`:

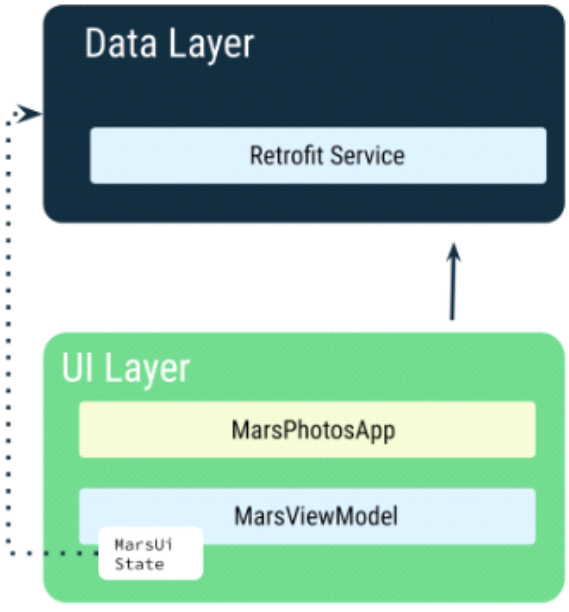
- This file contains the `HomeScreen` and `ResultScreen` composables. The `ResultScreen` has a simple `Box` layout that displays the value of `marsUiState` in a `Text` composable.

`MainActivity.kt`:

- The only task for this activity is to load the `ViewModel` and display the `MarsPhotosApp` composable.

4. Introduction to web services

In this codelab, you create a layer for the network service that communicates with the backend server and fetches the required data. You use a third-party library, called Retrofit, to implement this task. You learn more about this later. The `ViewModel` communicates with the data layer, and the rest of the app is transparent to this implementation.

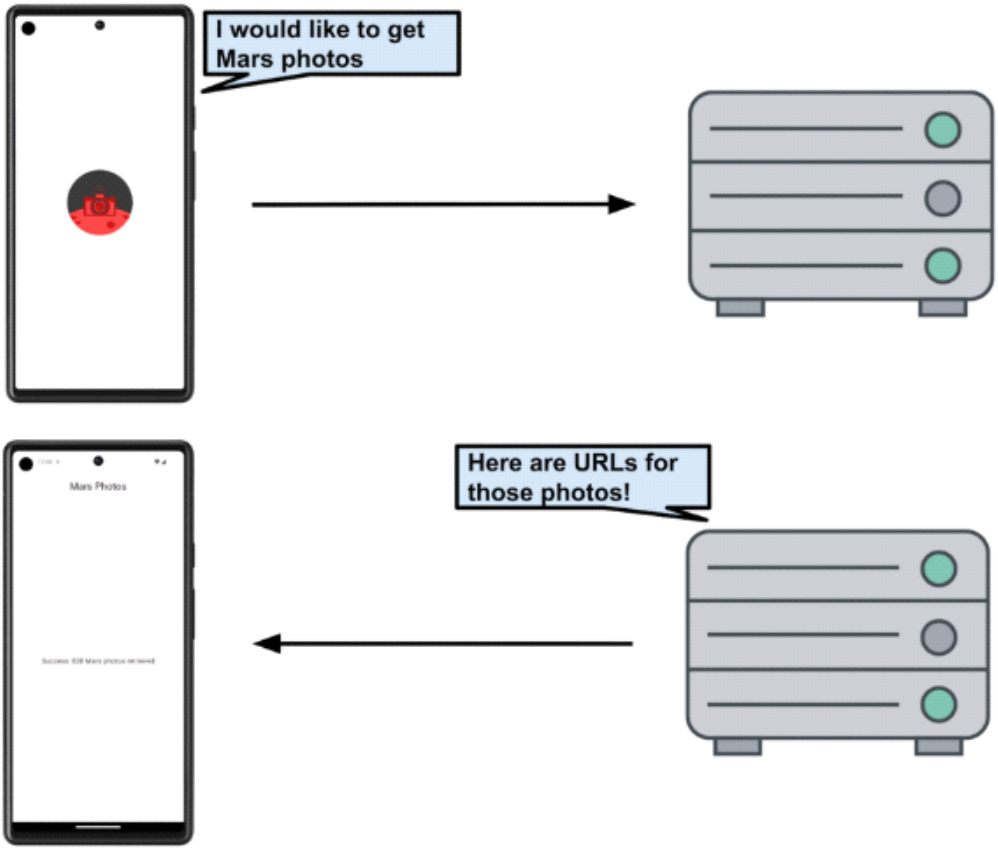


The `MarsViewModel` is responsible for making the network call to get the Mars photos data. In the `ViewModel`, you use `MutableState` to update the app UI when the data changes.

> **Note**: In later codelabs, you will add a Repository to your data layer. The Repository then communicates with the Retrofit service to fetch the data. The Repository is responsible for exposing the data to the rest of the app.

5. Web services and Retrofit

The Mars photos data is stored on a web server. To get this data into your app, you need to establish a connection and communicate with the server on the internet.



Note: In this codelab, you only retrieve the number of records (URLs), not the Mars photos. In a later codelab, you retrieve the Mars photos and display them in a grid.

Most web servers today run web services using a common stateless web architecture known as REST, which stands for **RE**presentational **S**tate **T**ransfer. Web services that offer this architecture are known as RESTful services.

Requests are made to RESTful web services in a standardized way, via Uniform Resource Identifiers (URIs). A URI identifies a resource in the server by name, without implying its location or how to access it. For example, in the app for this lesson, you retrieve the image URLs using the following server URI. (This server hosts both Mars real estate and Mars photos):

android-kotlin-fun-mars-server.appspot.com

Note: The aforementioned server is being accessed by a different sample app where it showcases Mars real estate, so this server has two different endpoints: one for Mars real estate and one for photos. For this course, you use the server to retrieve Mars photos.

you use the server to retrieve Mars photos.

A URL (Uniform Resource Locator) is a subset of a URI that specifies where a resource exists and the mechanism for retrieving it.

For example:

The following URL gets a list of available real estate properties on Mars:

https://android-kotlin-fun-mars-server.appspot.com/realestate

The following URL gets a list of Mars photos:

https://android-kotlin-fun-mars-server.appspot.com/photos

These URLs refer to an identified resource, such as /realestate or /photos, that is obtainable via the Hypertext Transfer Protocol (*http:*) from the network. You are using the /photos endpoint in this codelab. An endpoint is a URL that allows you to access a web service running on a server.

Note: The familiar web URL is actually a type of URI. This course uses both URL and URI interchangeably.

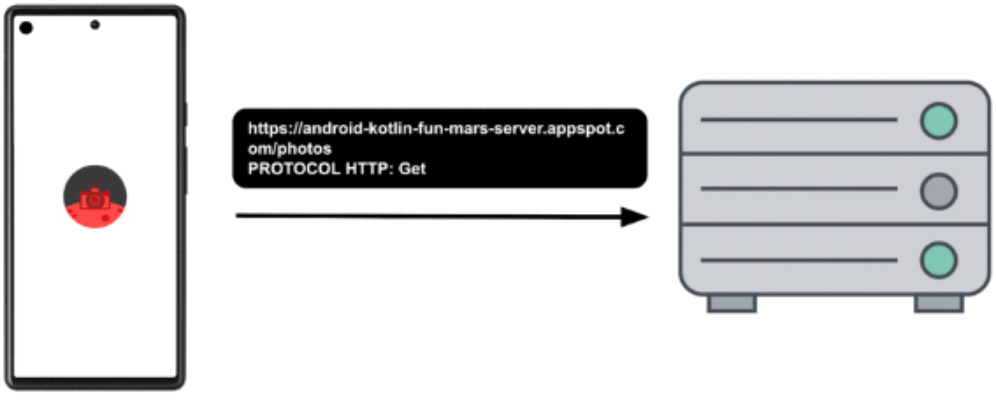
Web service request

Each web service request contains a URI and is transferred to the server using the same HTTP protocol that's used by web browsers, like Chrome. HTTP requests contain an operation to tell the server what to do.

Common HTTP operations include:

- GET for retrieving server data.
- POST for creating new data on the server.
- PUT for updating existing data on the server.
- DELETE for deleting data from the server.

Your app makes an HTTP GET request to the server for the Mars photos information, and then the server returns a response to your app, including the image URLs.



The response from a web service is formatted in one of the common data formats, like XML (eXtensible Markup Language) or JSON (JavaScript Object Notation). The JSON format represents structured data in key-value pairs. An app communicates with the REST API using JSON, which you learn more about in a later task.

In this task, you establish a network connection to the server, communicate with the server, and receive a JSON response. You will be using a backend server that is already written for you. In this codelab, you use the Retrofit library, a third-party library, to communicate with the backend server.

External Libraries

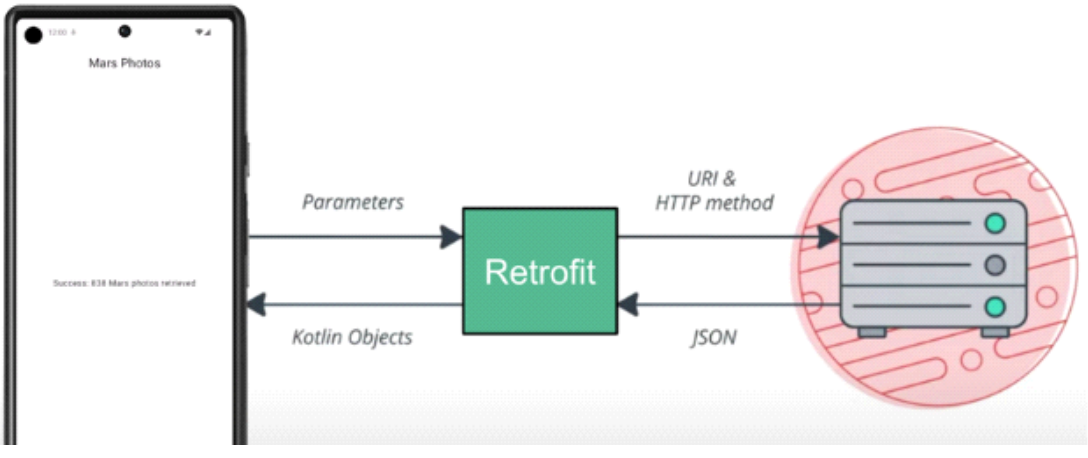
External libraries, or third-party libraries, are like extensions to the core Android APIs. The libraries you use in this course are open source, community-developed, and maintained by the collective contributions from the huge Android community around the world. These resources help Android developers like you to build better apps.

> **Warning**: Using community-developed and maintained libraries can be a huge timesaver. However, you must choose these libraries wisely because your app is ultimately responsible for what the code does in these libraries.

Retrofit Library

The Retrofit library that you use in this codelab to talk to the RESTful Mars web service is a good example of a well-supported and maintained library. You can tell this by looking at its GitHub page and reviewing the open and closed issues (some are feature requests). If the developers are regularly resolving the issues and responding to the feature requests, the library is likely well-maintained and a good candidate to use in the app. You can also refer to Retrofit documentation to learn more about the library.

The Retrofit library communicates with the REST backend. It generates the code, but you need to provide the URIs for the web service based on the parameters we pass to it. You learn more about this topic in later sections.



Add Retrofit dependencies

Android Gradle lets you add external libraries to your project. In addition to the library dependency, you also need to include the repository where the library is hosted.

1. Open the module-level gradle file `build.gradle.kts (Module :app)`.
2. In the `dependencies` section, add the following lines for the Retrofit libraries:

```
// Retrofit
implementation("com.squareup.retrofit2:retrofit:2.9.0")
// Retrofit with Scalar Converter
implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
```

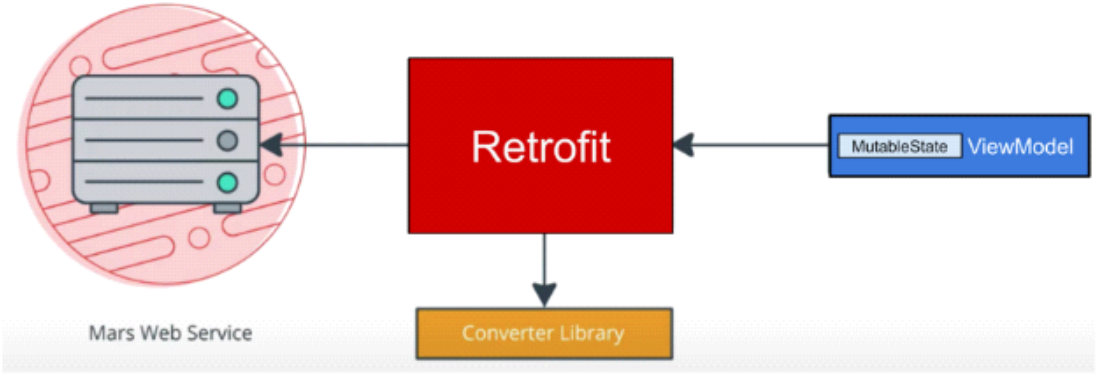
The two libraries work together. The first dependency is for the Retrofit2 library itself, and the second dependency is for the Retrofit scalar converter. Retrofit2 is the updated version of the Retrofit library. This scalar converter enables Retrofit to return the JSON result as a `String`. JSON is a format for storing and transporting data between client and server. You will learn about JSON in a later section.

3. Click **Sync Now** to rebuild the project with the new dependencies.

6. Connecting to the Internet

You use the Retrofit library to talk to the Mars web service and display the raw JSON response as a `String`. The placeholder `Text` either displays the returned JSON response string or a message indicating a connection error.

Retrofit creates a network API for the app based on the content from the web service. It fetches data from the web service and routes it through a separate converter library that knows how to decode the data and return it in the form of objects, like `String`. Retrofit includes built-in support for popular data formats, such as XML and JSON. Retrofit ultimately creates the code to call and consume this service for you, including critical details, such as running the requests on background threads.



In this task, you add a data layer to your **Mars Photos** project that your `ViewModel` uses to communicate with the

In this task, you add a data layer to your **Mars Photos** project that your `ViewModel` uses to communicate with the web service. You implement the Retrofit service API with the following steps:

- Create a data source, `MarsApiService` class.
- Create a Retrofit object with the base URL and the converter factory to convert strings.
- Create an interface that explains how Retrofit talks to the web server.
- Create a Retrofit service and expose the instance to the api service to the rest of the app.

Implement the above steps:

1. Right-click on the package com.example.marsphotos in your Android project pane and select **New > Package**.
2. In the popup, append **network** to the end of the suggested package name.
3. Create a new Kotlin file under the new package. Name it `MarsApiService`.
4. Open `network/MarsApiService.kt`.
5. Add the following constant for the base URL for the web service.

```
private const val BASE_URL =
    "https://android-kotlin-fun-mars-server.appspot.com"
```

6. Add a Retrofit builder just below that constant to build and create a Retrofit object.

```
import retrofit2.Retrofit

private val retrofit = Retrofit.Builder()
```

Retrofit needs the base URI for the web service and a converter factory to build a web services API. The converter tells Retrofit what to do with the data it gets back from the web service. In this case, you want Retrofit to fetch a JSON response from the web service and return it as a `String`. Retrofit has a `ScalarsConverter` that supports strings and other primitive types.

7. Call `addConverterFactory()` on the builder with an instance of `ScalarsConverterFactory`.

```
import retrofit2.converter.scalars.ScalarsConverterFactory

private val retrofit = Retrofit.Builder()
    .addConverterFactory(ScalarsConverterFactory.create())
```

8. Add the base URL for the web service using the `baseUrl()` method.
9. Call `build()` to create the Retrofit object.

```
private val retrofit = Retrofit.Builder()
    .addConverterFactory(ScalarsConverterFactory.create())
    .baseUrl(BASE_URL)
    .build()
```

```
    .addConverterFactory(ScalarsConverterFactory.create())
    .baseUrl(BASE_URL)
    .build()
```

10. Below the call to the Retrofit builder, define an interface called `MarsApiService` that defines how Retrofit talks to the web server using HTTP requests.

```
interface MarsApiService {
}
```

11. Add a function called `getPhotos()` to the `MarsApiService` interface to get the response string from the web service.

```
interface MarsApiService {
    fun getPhotos()
}
```

12. Use the `@GET` annotation to tell Retrofit that this is a GET request and specify an endpoint for that web service method. In this case, the endpoint is `photos`. As mentioned in the previous task, you will use the /photos endpoint in this codelab.

```
import retrofit2.http.GET


interface MarsApiService {
    @GET("photos")
    fun getPhotos()
}
```

When the `getPhotos()` method is invoked, Retrofit appends the endpoint `photos` to the base URL—which you defined in the Retrofit builder—used to start the request.

13. Add a return type of the function to `String`.

```
interface MarsApiService {
    @GET("photos")
    fun getPhotos(): String
}
```

Object declarations

In Kotlin, object declarations are used to declare singleton objects. Singleton pattern ensures that one, and only one, instance of an object is created and has one global point of access to that object. Object initialization is thread-safe and done at first access.

Following is an example of an object declaration and its access. Object declaration always has a name following the `object` keyword

Following is an example of an object declaration and its access. Object declaration always has a name following the `object` keyword.

Example:

```
// Example for Object declaration, do not copy over

object SampleDataProvider {
    fun register(provider: SampleProvider) {
        // ...
    }

    // ...
}

// To refer to the object, use its name directly.
SampleDataProvider.register(...)
```

> **Warning**: Singleton pattern is not a recommended practice. Singletons represent global states that are hard to predict, particularly in tests. Objects should define which dependencies they need, instead of describing how to create them.
>
> Use Dependency injection over singleton pattern, you will learn how to implement Dependency injection in a later codelab.

The call to `create()` function on a Retrofit object is expensive in terms of memory, speed, and performance. The app needs *only one* instance of the Retrofit API service, so you expose the service to the rest of the app using *object declaration*.

1. Outside the `MarsApiService` interface declaration, define a public object called `MarsApi` to initialize the Retrofit service. This object is the public singleton object that the rest of the app can access.

```
object MarsApi {}
```

2. Inside the `MarsApi` object declaration, add a lazily initialized retrofit object property named `retrofitService` of the type `MarsApiService`. You make this lazy initialization to make sure it is initialized at its first usage. Ignore the error, which you fix in the next steps.

> **Note**: Remember "lazy initialization" is when object creation is purposely delayed, until you actually need that object, to avoid unnecessary computation or use of other computing resources. Kotlin has first-class support for lazy instantiation.

```
object MarsApi {
    val retrofitService : MarsApiService by lazy {}
}
```

```
    val retrofitService : MarsApiService by lazy {}
}
```

3. Initialize the `retrofitService` variable using the `retrofit.create()` method with the `MarsApiService` interface.

```
object MarsApi {
    val retrofitService : MarsApiService by lazy {
       retrofit.create(MarsApiService::class.java)
    }
}
```

The Retrofit setup is done! Each time your app calls `MarsApi.retrofitService`, the caller accesses the same singleton Retrofit object that implements `MarsApiService`, which is created on the first access. In the next task, you use the Retrofit object you implemented.

Call the web service in MarsViewModel

In this step, you implement the `getMarsPhotos()` method that calls the REST service and then handles the returned JSON string.

> **Note**: The recommended approach is to call webservice from a repository, which isolates the data layer from the rest of the app. In a later codelab, you add a repository to your app.

ViewModelScope

A `viewModelScope` is the built-in coroutine scope defined for each `ViewModel` in your app. Any coroutine launched in this scope is automatically canceled if the `ViewModel` is cleared.

You can use `viewModelScope` to launch the coroutine and make the web service request in the background. Since the `viewModelScope` belongs to the `ViewModel`, the request continues even if the app goes through a configuration change.

1. In the `MarsApiService.kt` file, make `getPhotos()` a suspend function to make it asynchronous and not block the calling thread. You call this function from inside a `viewModelScope`.

```
@GET("photos")
suspend fun getPhotos(): String
```

2. Open the `ui/screens/MarsViewModel.kt` file. Scroll down to the `getMarsPhotos()` method. Delete the line that sets the status response to `"Set the Mars API Response here!"` so that the method `getMarsPhotos()` is empty.

```
private fun getMarsPhotos() {}
```

```
private fun getMarsPhotos() {}
```

3. Inside `getMarsPhotos()`, launch the coroutine using `viewModelScope.launch`.

```
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.launch


private fun getMarsPhotos() {
    viewModelScope.launch {}
}
```

4. Inside `viewModelScope`, use the singleton object `MarsApi` to call the `getPhotos()` method from the `retrofitService` interface. Save the returned response in a `val` called `listResult`.

```
import com.example.marsphotos.network.MarsApi

viewModelScope.launch {
    val listResult = MarsApi.retrofitService.getPhotos()
}
```

5. Assign the result just received from the backend server to the `marsUiState`. The `marsUiState` is a mutable state object that represents the status of the most recent web request.

```
val listResult = MarsApi.retrofitService.getPhotos()
marsUiState = listResult
```

6. Run the app. Notice that the app closes immediately, and it may or may not display an error popup. This is an app crash.
7. Click the **Logcat** tab in Android Studio and note the error in the log, which starts with a line like this: "`--------- beginning of crash`"

```
    --------- beginning of crash
22803-22865/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION: OkHttp Disp
    Process: com.example.android.marsphotos, PID: 22803
    java.lang.SecurityException: Permission denied (missing INTERNET permission?)
...
```

This error message indicates the app might be missing the `INTERNET` permissions. The next task describes how to add internet permissions to the app and resolve this issue.

7. Add Internet permission and Exception Handling

Android Permissions

Android Permissions

The purpose of permissions on Android is to protect the privacy of an Android user. Android apps must declare or request permissions to access sensitive user data, such as contacts, call logs, and certain system features, such as camera or internet.

In order for your app to access the Internet, it needs the `INTERNET` permission. Connecting to the internet introduces security concerns, which is why apps do not have internet connectivity by default. You need to explicitly declare that the app needs access to the internet. This declaration is considered a normal permission. To learn more about Android permissions and its types, please refer to the Permissions on Android.

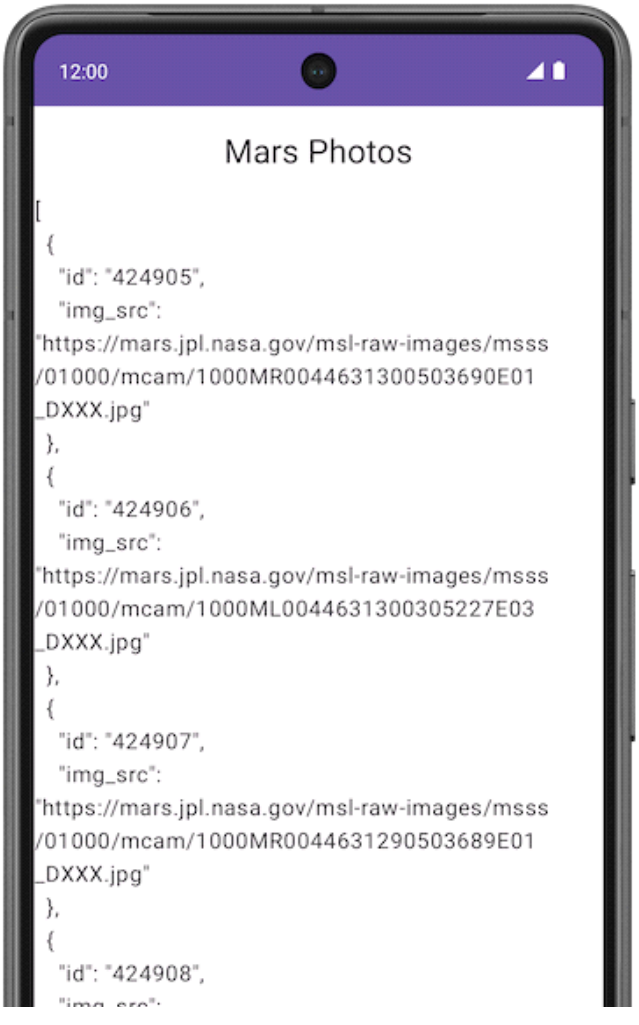
In this step, your app declares the permission(s) it requires by including `<uses-permission>` tags in the `AndroidManifest.xml` file.

1. Open `manifests/AndroidManifest.xml`. Add this line just before the `<application>` tag:

```
<uses-permission android:name="android.permission.INTERNET" />
```

2. Compile and run the app again.

If you have a working internet connection, you see the JSON text containing data related to the Mars photos. Observe how the `id` and `img_src` are repeated for every image record. You learn more about the JSON format later in the codelab.



3. Tap the **Back** button in your device or emulator to close the app.

Exception Handling

There is a bug in your code. Perform the following steps to see it:

1. Put your device or emulator into Airplane Mode to simulate a network connection error.
2. Reopen the app from the Recents menu, or run the app from Android Studio.
3. Click the **Logcat** tab in Android Studio and note the fatal exception in the log, which looks like the following:

```
3302-3302/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION: main
    Process: com.example.android.marsphotos, PID: 3302
```

This error message indicates the application tried to connect and timed out. Exceptions like this one are very common in real time. Unlike the permission issue, this error is not something you can fix, but you can handle it. In the next step, you learn how to handle such exceptions.

Exceptions

Exceptions are errors that can occur during runtime, not compile time, and they terminate the app abruptly without notifying the user. This can result in a poor user experience. *Exception handling* is a mechanism by which you prevent the app from terminating abruptly and handle the situation in a user-friendly way.

The reason for exceptions could be as simple as division by zero or an error with the network connection. These exceptions are similar to the `IllegalArgumentException` that a previous codelab discusses.

Examples of potential issues while connecting to a server include the following:

- The URL or URI used in the API is incorrect.
- The server is unavailable, and the app could not connect to it.
- A network latency issue.
- Poor or no internet connection on the device.

These exceptions can't be handled during compile time, but you can use a `try-catch` block to handle the exception in runtime. For further learning, refer to Exceptions.

Example syntax for try-catch block

Example syntax for try-catch block

```
try {
    // some code that can cause an exception.
}
catch (e: SomeException) {
    // handle the exception to avoid abrupt termination.
}
```

Inside the `try` block, you add the code where you anticipate an exception. In your app, this is a network call. In the `catch` block, you need to implement the code that prevents abrupt termination of the app. If there is an exception, then the `catch` block executes to recover from the error instead of terminating the app abruptly.

1. In `getMarsPhotos()`, inside the `launch` block, add a `try` block around the `MarsApi` call to handle exceptions.
2. Add a `catch` block after the `try` block.

```
import java.io.IOException


viewModelScope.launch {
   try {
       val listResult = MarsApi.retrofitService.getPhotos()
       marsUiState = listResult
   } catch (e: IOException) {

   }
}
```

3. Run the app one more time. Notice that the app does not crash this time.

Add State UI

In the `MarsViewModel` class, the status of the most recent web request, `marsUiState`, is saved as a mutable state object. However, this class lacks the ability to save the different status: loading, success, and failure.

- **Loading** status indicates the app is waiting for data.
- **Success** status indicates the data was successfully retrieved from the web service.
- **Error** status indicates any network or connection errors.

To represent these three states in your application, you use a *sealed interface*. A `sealed interface` makes it easy to manage state by limiting the possible values. In the Mars Photos app, you restrict the `marsUiState` web response to three states (data class objects): loading, success, and error, which looks like the following code:

```
// No need to copy over
sealed interface MarsUiState {
   data class Success : MarsUiState
   data class Loading : MarsUiState
   data class Error : MarsUiState
}
```

```
    data class Loading : MarsUiState
    data class Error : MarsUiState
}
```

In the above code snippet, in the case of a successful response, you receive Mars photo information from the server. In order to store the data, add a constructor parameter to the `Success` data class.

In the case of `Loading` and `Error` states, you don't need to set new data and create new objects; you are just passing the web response. Change the `data` class to `Object` to create the objects for the web responses.

1. Open the `ui/MarsViewModel.kt` file. After the import statements, add the `MarsUiState` sealed interface. This addition makes the values `MarsUiState` object can have exhaustive.

```
sealed interface MarsUiState {
    data class Success(val photos: String) : MarsUiState
    object Error : MarsUiState
    object Loading : MarsUiState
}
```

2. Inside the `MarsViewModel` class, update the `marsUiState` definition. Change the type to `MarsUiState` and `MarsUiState.Loading` as its default value. Make the setter private to protect writes to the `marsUiState`.

```
var marsUiState: MarsUiState by mutableStateOf(MarsUiState.Loading)
  private set
```

3. Scroll down to the `getMarsPhotos()` method. Update the `marsUiState` value to `MarsUiState.Success` and pass the `listResult`.

```
val listResult = MarsApi.retrofitService.getPhotos()
marsUiState = MarsUiState.Success(listResult)
```

4. Inside the `catch` block, handle the failure response. Set `MarsUiState` to `Error`.

```
catch (e: IOException) {
    marsUiState = MarsUiState.Error
}
```

5. You can lift the `marsUiState` assignment out of the `try-catch` block. Your completed function should look like the following code:

```
private fun getMarsPhotos() {
    viewModelScope.launch {
        marsUiState = try {
            val listResult = MarsApi.retrofitService.getPhotos()
            MarsUiState.Success(listResult)
        } catch (e: IOException) {
```

```
            val listResult = MarsApi.retrofitService.getPhotos()
            MarsUiState.Success(listResult)
        } catch (e: IOException) {
            MarsUiState.Error
        }
    }
}
```

6. In the `screens/HomeScreen.kt` file, add a `when` expression on the `marsUiState`. If the `marsUiState` is `MarsUiState.Success`, call `ResultScreen` and pass in `marsUiState.photos`. Ignore the errors for now.

```
import androidx.compose.foundation.layout.fillMaxWidth

fun HomeScreen(
    marsUiState: MarsUiState,
    modifier: Modifier = Modifier
) {
    when (marsUiState) {
        is MarsUiState.Success -> ResultScreen(
            marsUiState.photos, modifier = modifier.fillMaxWidth()
        )
    }
}
```

Note: The `marsUiState` property is not a string any longer. You changed it to a `MarsUiState` sealed interface, which can have three different object values: `MarsUiState.Loading`, `MarsUiState.Success`, and `MarsUiState.Error`.

7. Inside the `when` block, add checks for `MarsUiState.Loading` and `MarsUiState.Error`. Have the app display the `LoadingScreen`, `ResultScreen`, and `ErrorScreen` composables, which you implement later.

```
import androidx.compose.foundation.layout.fillMaxSize

fun HomeScreen(
    marsUiState: MarsUiState,
    modifier: Modifier = Modifier
) {
    when (marsUiState) {
        is MarsUiState.Loading -> LoadingScreen(modifier = modifier.fillMaxSize())
        is MarsUiState.Success -> ResultScreen(
            marsUiState.photos, modifier = modifier.fillMaxWidth()
        )

        is MarsUiState.Error -> ErrorScreen( modifier = modifier.fillMaxSize())
    }
}
```

Note: If you implement `MarsUiState interface` without a `sealed` keyword, it requires you to add a Success, Error, Loading and `else` branch. Since there is no fourth option (else), you use a `sealed` interface to tell the

Note: If you implement `MarsUiState interface` without a `sealed` keyword, it requires you to add a Success, Error, Loading and `else` branch. Since there is no fourth option (else), you use a `sealed` interface to tell the compiler that there are only three options (thus making the conditionals exhaustive).

8. Open `res/drawable/loading_animation.xml`. This drawable is an animation that rotates an image drawable, `loading_img.xml`, around the center point. (You don't see the animation in the preview.)

9. In the `screens/HomeScreen.kt` file, below the `HomeScreen` composable, add the following `LoadingScreen` composable function to display the loading animation. The `loading_img` drawable resource is included in the starter code.

```
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.Image


@Composable
fun LoadingScreen(modifier: Modifier = Modifier) {
    Image(
        modifier = modifier.size(200.dp),
        painter = painterResource(R.drawable.loading_img),
        contentDescription = stringResource(R.string.loading)
    )
}
```

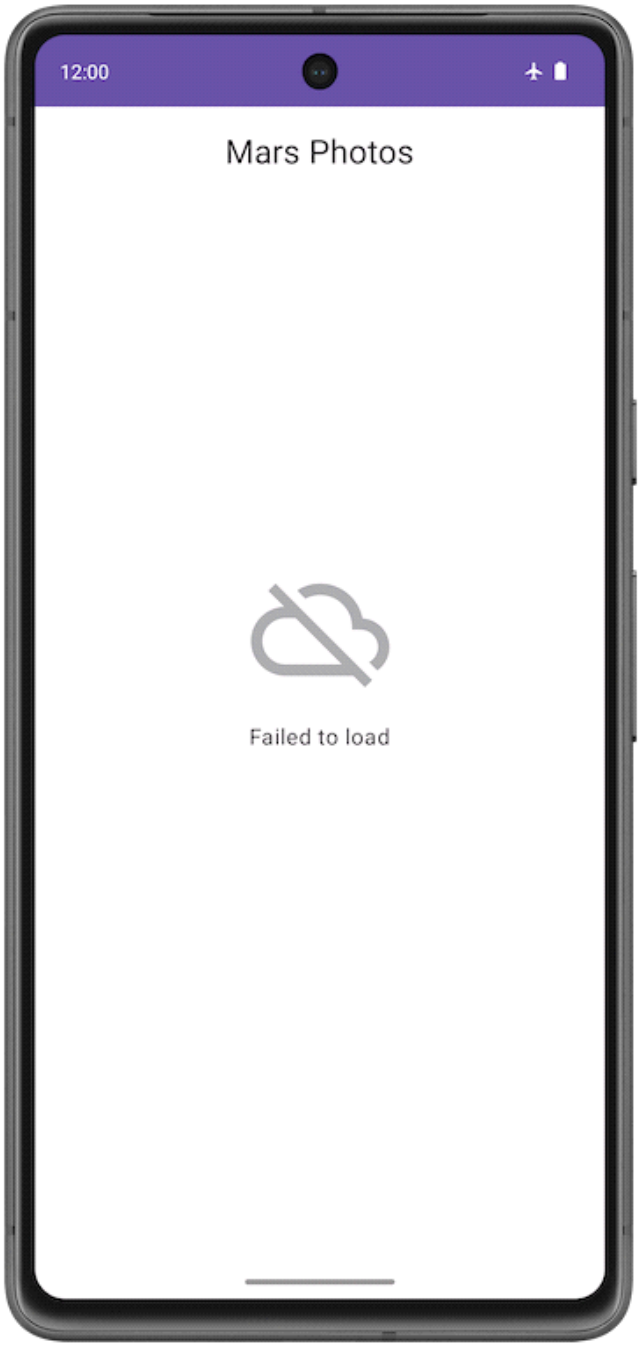
10. Below the `LoadingScreen` composable, add the following `ErrorScreen` composable function so the app can display the error message.

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding

@Composable
fun ErrorScreen(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier,
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Image(
            painter = painterResource(id = R.drawable.ic_connection_error), contentDescr
        )
        Text(text = stringResource(R.string.loading_failed), modifier = Modifier.padding(
    }
}
```

```
    }
}
```

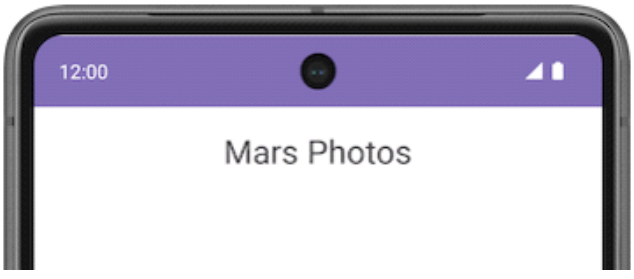
11. Run the app again with Airplane Mode turned on. The app does not close abruptly this time, and it displays the following error message:

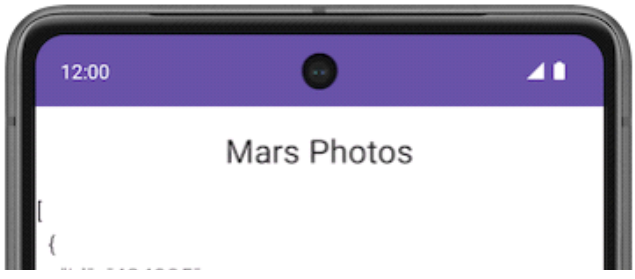


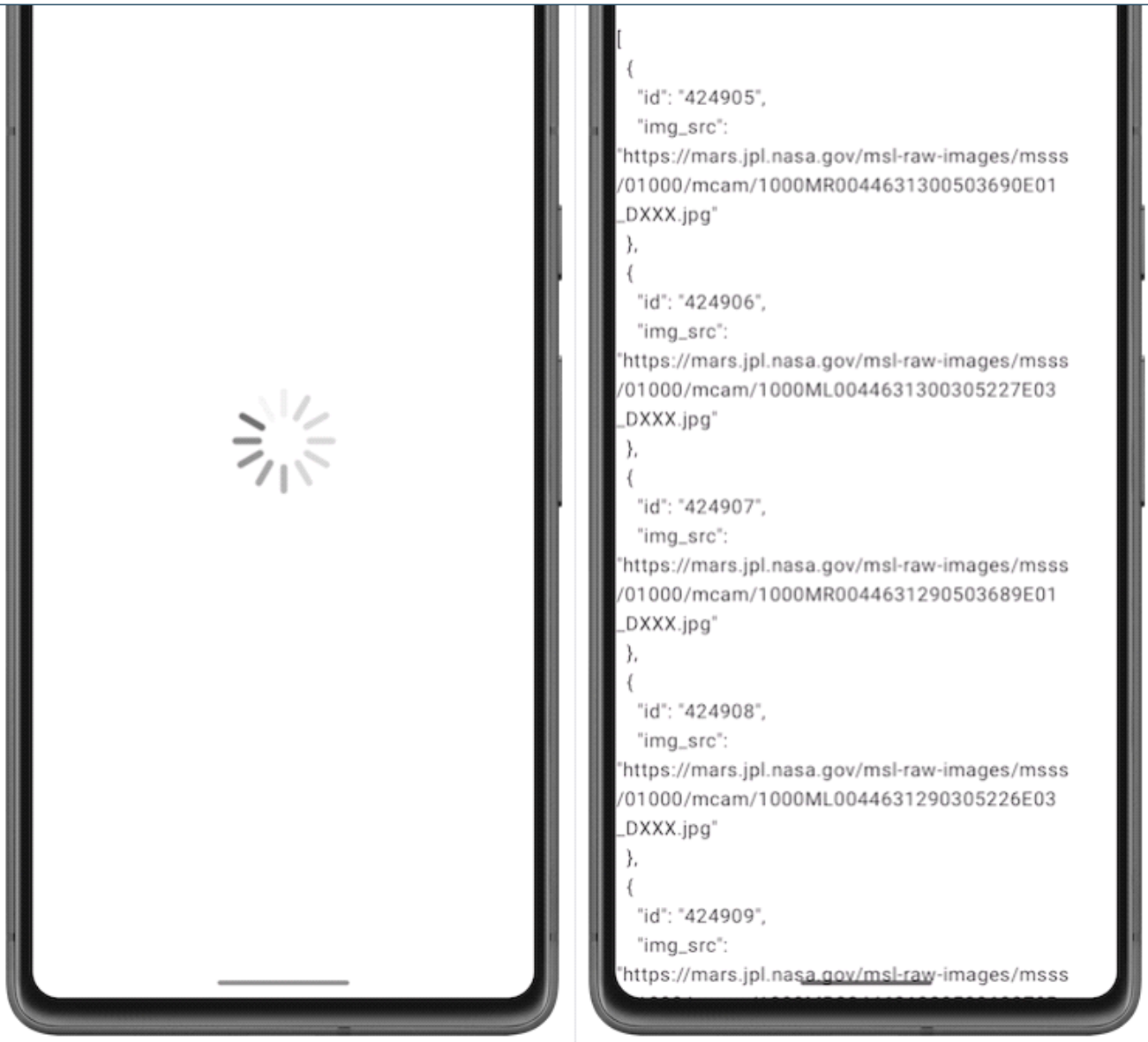
Back

Next

12. Turn off Airplane Mode on your phone or emulator. Run and test your app to make sure everything works correctly and you are able to see the JSON string.







8. Parse the JSON response with kotlinx.serialization

JSON

The requested data is typically formatted in one of the common data formats like XML or JSON. Each call returns structured data, and your app needs to know what that structure is in order to read the data from the response.

For example, in this app, you are retrieving the data from the https:// android-kotlin-fun-mars-server.appspot.com/photos server. When you enter this URL in the browser, you see a list of IDs and image URLs of the surface of Mars in a JSON format!

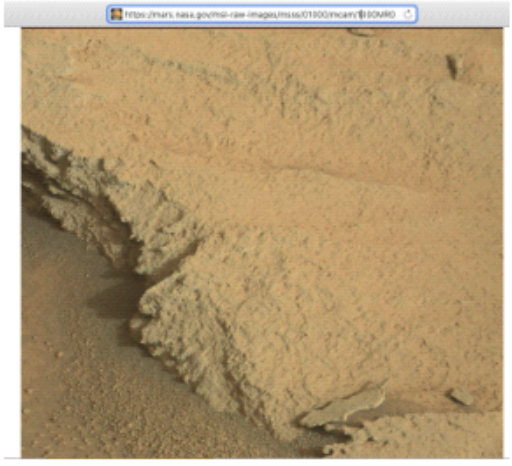
Structure of sample JSON response

JSON array structure

The structure of a JSON response has the following features:

- JSON response is an array, indicated by the square brackets. The array contains JSON objects.
- JSON objects are surrounded by curly brackets.
- Each JSON object contains a set of key-value pairs separated by a comma.
- A colon separates the key and value in a pair.
- Names are surrounded by quotes.
- Values can be numbers, strings, a boolean, an array, an object (JSON object), or null.

For example, the `img_src` is a URL, which is a string. When you paste the URL into a web browser, you see a Mars surface image.



In your app, now you're getting a JSON response from the Mars web service, which is a great start. But what you really need to display the images are Kotlin objects, not a big JSON string. This process is called *deserialization*.

Serialization is the process of converting data used by an application to a format that can be transferred over a network. As opposed to *serialization*, *deserialization* is the process of reading data from an external source (like a server) and converting it into a runtime object. They are both essential components of most applications that exchange data over the network.

The `kotlinx.serialization` provides sets of libraries that convert a JSON string into Kotlin objects. There is a community developed third party library that works with Retrofit, Kotlin Serialization Converter.

In this task, you use the `kotlinx.serialization` library, to parse the JSON response from the web service into useful Kotlin objects that represent Mars photos. You change the app so that instead of displaying the raw JSON, the app displays the number of Mars photos returned.

Add `kotlinx.serialization` library dependencies

1. Open `build.gradle.kts (Module :app)`.
2. In the `plugins` block, add `kotlinx serialization` plugin.

2. In the `plugins` block, add `kotlinx serialization` plugin.

```
id("org.jetbrains.kotlin.plugin.serialization") version "1.8.10"
```

3. In the `dependencies` section, add the following code to include the `kotlinx.serialization` dependency. This dependency provides JSON serialization for Kotlin projects.

```
// Kotlin serialization
implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.5.1")
```

4. Locate the lines for the Retrofit scalar converter in the `dependencies` block and change it to use `kotlinx-serialization-converter`:

Replace the following code

```
// Retrofit with scalar Converter
implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
```

with the following code

```
// Retrofit with Kotlin serialization Converter

implementation("com.jakewharton.retrofit:retrofit2-kotlinx-serialization-converter:1.0.0
implementation("com.squareup.okhttp3:okhttp:4.11.0")
```

5. Click **Sync Now** to rebuild the project with the new dependencies.

> **Note:** The project may show compiler errors related to the removed Retrofit scalar dependency. You fix those in the next steps.

Implement the Mars Photo data class

A sample entry of the JSON response you get from the web service looks something like the following, similar to what you saw earlier:

```
[
    {
        "id":"424906",
        "img_src":"http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631
    },
...]
```

In the example above, notice that each Mars photo entry has the following JSON key and value pairs:

In the example above, notice that each Mars photo entry has the following JSON key and value pairs:

- `id`: the ID of the property, as a string. Since it is wrapped in quotes (`" "`), it is of the type `String`, not `Integer`.
- `img_src`: The image's URL, as a string.

kotlinx.serialization parses this JSON data and converts it into Kotlin objects. To do this, kotlinx.serialization needs to have a Kotlin data class to store the parsed results. In this step, you create the data class `MarsPhoto`.

1. Right-click on the **network** package and select **New > Kotlin File/Class**.
2. In the dialog, select **Class** and enter `MarsPhoto` as the name of the class. This action creates a new file called `MarsPhoto.kt` in the `network` package.
3. Make `MarsPhoto` a data class by adding the `data` keyword before the class definition.
4. Change the curly braces `{}` to parentheses `()`. This change gives you an error because data classes must have at least one property defined.

```
data class MarsPhoto()
```

5. Add the following properties to the `MarsPhoto` class definition.

```
data class MarsPhoto(
    val id: String,  val img_src: String
)
```

6. To make `MarsPhoto` class serializable annotating it with `@Serializable`.

```
import kotlinx.serialization.Serializable

@Serializable
data class MarsPhoto(
    val id: String,  val img_src: String
)
```

Notice that each of the variables in the `MarsPhoto` class corresponds to a key name in the JSON object. To match the types in our specific JSON response, you use `String` objects for all the values.

When `kotlinx serialization` parses the JSON, it matches the keys by name and fills the data objects with appropriate values.

@SerialName Annotation

Sometimes the key names in a JSON response can make confusing Kotlin properties or may not match recommended coding style. For example, in the JSON file, the `img_src` key uses an underscore, whereas Kotlin convention for properties uses upper and lowercase letters (camel case).

To use variable names in your data class that differ from the key names in the JSON response, use the `@SerialName`

properties uses upper and lowercase letters (camel case).

To use variable names in your data class that differ from the key names in the JSON response, use the `@SerialName` annotation. In the following example, the name of the variable in the data class is `imgSrc`. The variable can be mapped to the JSON attribute `img_src` using `@SerialName(value = "img_src")`.

1. Replace the line for the `img_src` key with the line shown below.

```
import kotlinx.serialization.SerialName

@SerialName(value = "img_src")
val imgSrc: String
```

Update MarsApiService and MarsViewModel

In this task, you will use the `kotlinx.serialization` converter to convert the JSON object to Kotlin objects.

1. Open `network/MarsApiService.kt`.
2. Notice the unresolved reference errors for `ScalarsConverterFactory`. These errors are a result of the Retrofit dependency change in a previous section.
3. Delete the import for `ScalarConverterFactory`. You fix the other error later.

Remove:

```
import retrofit2.converter.scalars.ScalarsConverterFactory
```

4. In the `retrofit` object declaration, change the Retrofit builder to use the `kotlinx.serialization` instead of the `ScalarConverterFactory`.

```
import com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
import kotlinx.serialization.json.Json
import okhttp3.MediaType

private val retrofit = Retrofit.Builder()
        .addConverterFactory(Json.asConverterFactory("application/json".toMediaType()))
        .baseUrl(BASE_URL)
        .build()
```

Note: If Android Studio shows you any warning regarding `kotlinx.serialization.ExperimentalSerializationApi` ignore it for now. You don't have to worry about this warning. It will be resolved in the later versions of Kotlin.

Now that you have the `kotlinx.serialization` in place, you can ask Retrofit to return a list of `MarsPhoto` objects

Now that you have the `kotlinx.serialization` in place, you can ask Retrofit to return a list of `MarsPhoto` objects from the JSON array, instead of returning a JSON string.

5. Update the `MarsApiService` interface for Retrofit to return a list of `MarsPhoto` objects instead of returning a `String`.

```
interface MarsApiService {
    @GET("photos")
    suspend fun getPhotos(): List<MarsPhoto>
}
```

6. Make similar changes to the `viewModel`. Open `MarsViewModel.kt` and scroll down to the `getMarsPhotos()` method.

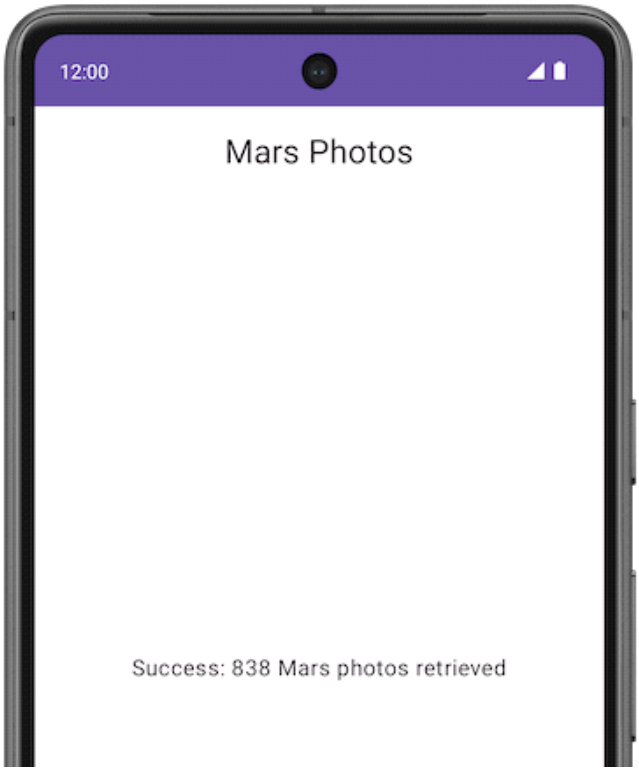
In the `getMarsPhotos()` method, `listResult` is a `List<MarsPhoto>` and not a `String` anymore. The size of that list is the number of photos that were received and parsed.

7. To print the number of photos retrieved, update `marsUiState` as follows:

```
val listResult = MarsApi.retrofitService.getPhotos()
marsUiState = MarsUiState.Success(
   "Success: ${listResult.size} Mars photos retrieved"
)
```

8. Make sure Airplane Mode is turned *off* on your device or emulator. Compile and run the app.

This time, the message should show the number of properties returned from the web service, and not a big JSON string:



Note: If your internet connection is not working, make sure that you turn off Airplane Mode on your device or emulator.

9. Solution code

To download the code for the finished codelab, you can use these git commands:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-tr
$ cd basic-android-kotlin-compose-training-mars-photos
$ git checkout repo-starter
```

Alternatively, you can download the repository as a zip file, unzip it, and open it in Android Studio.

Download zip

Note: The solution code is in the `repo-starter` branch of the downloaded repository.

If you want to see the solution code for this codelab, view it on GitHub.

$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-mars-photos.git
$ cd basic-android-kotlin-compose-training-mars-photos
$ git checkout repo-starter

basic and

$ cd basic-android-kotlin-compose-training-mars-photos
$ git checkout repo-starter

basic-and
roid-kotli...

10. Summary

REST web services

- A *web service* is software-based functionality, offered over the internet, that enables your app to make requests and get data back.
- Common web services use a REST architecture. Web services that offer REST architecture are known as *RESTful* services. RESTful web services are built using standard web components and protocols.
- You make a request to a REST web service in a standardized way via URIs.
- To use a web service, an app must establish a network connection and communicate with the service. Then the app must receive and parse response data into a format the app can use.
- The Retrofit library is a client library that enables your app to make requests to a REST web service.
- Use converters to tell Retrofit what to do with the data it sends to the web service and gets back from the web service. For example, the `ScalarsConverter` treats the web service data as a `String` or other primitive.
- To enable your app to make connections to the internet, add the `"android.permission.INTERNET"` permission in the Android manifest.
- Lazy initialization delegates the creation of an object to the first time it is used. It creates the reference but not the object. When an object is accessed for the first time, a reference is created and used every time thereafter.

JSON parsing

- The response from a web service is often formatted in JSON, a common format to represent structured data.
- A JSON object is a collection of key-value pairs.
- A collection of JSON objects is a JSON array. You get a JSON array as a response from a web service.
- The keys in a key-value pair are surrounded by quotes. The values can be numbers or strings.
- In Kotlin, data serialization tools are available in a separate component, *kotlinx.serialization*. The *kotlinx.serialization* provides sets of libraries that convert a JSON string into Kotlin objects.
- There is a community developed Kotlin Serialization Converter library for Retrofit: retrofit2-kotlinx-serialization-converter.
- The *kotlinx.serialization* matches the keys in a JSON response with properties in a data object that have the same name.
- To use a different property name for a key, annotate that property with the `@SerialName` annotation and the JSON key `value`.